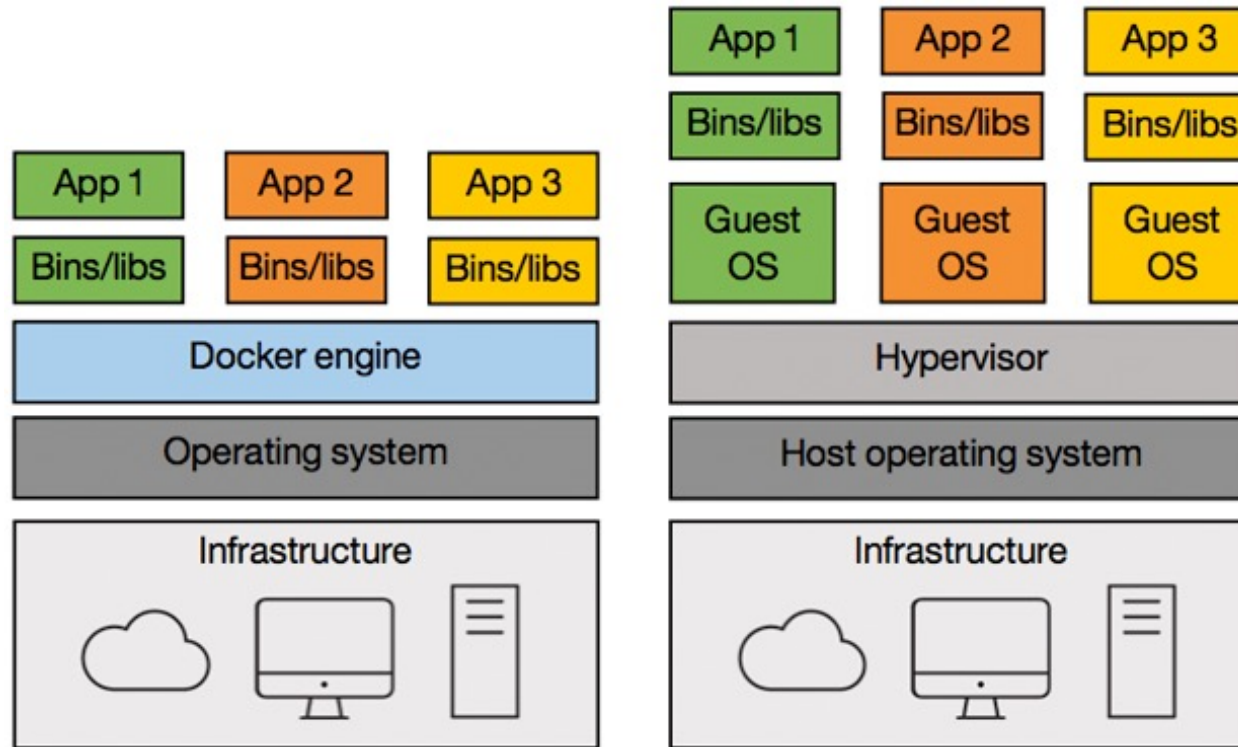




VIRTUALISATION PAR CONTENEURS

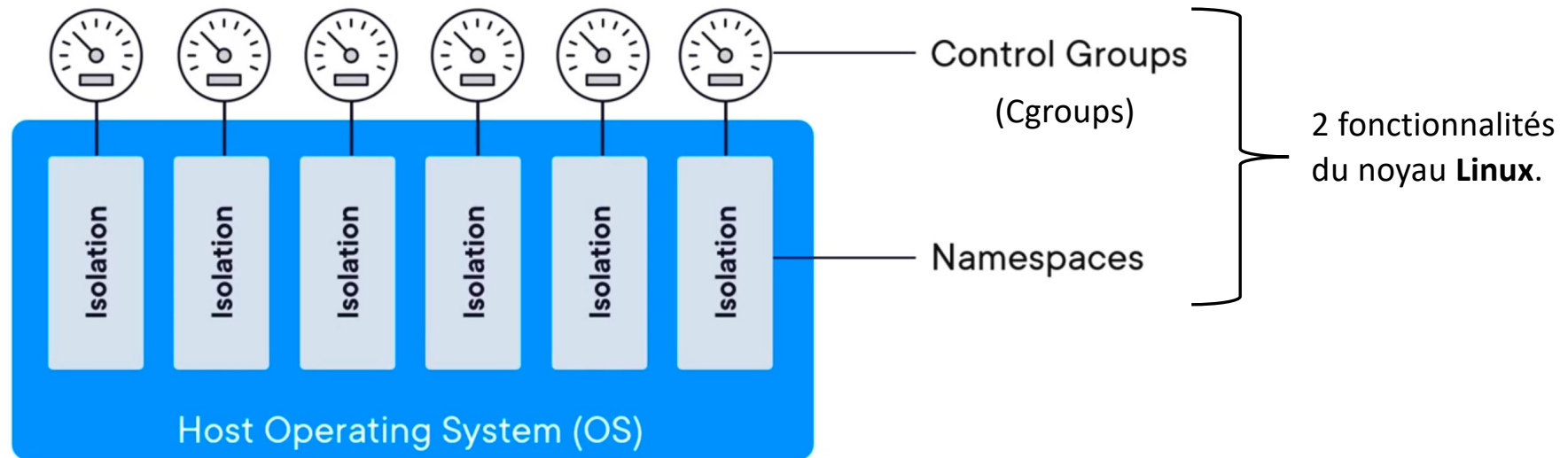
Conteneur vs VM



Conteneur

Container

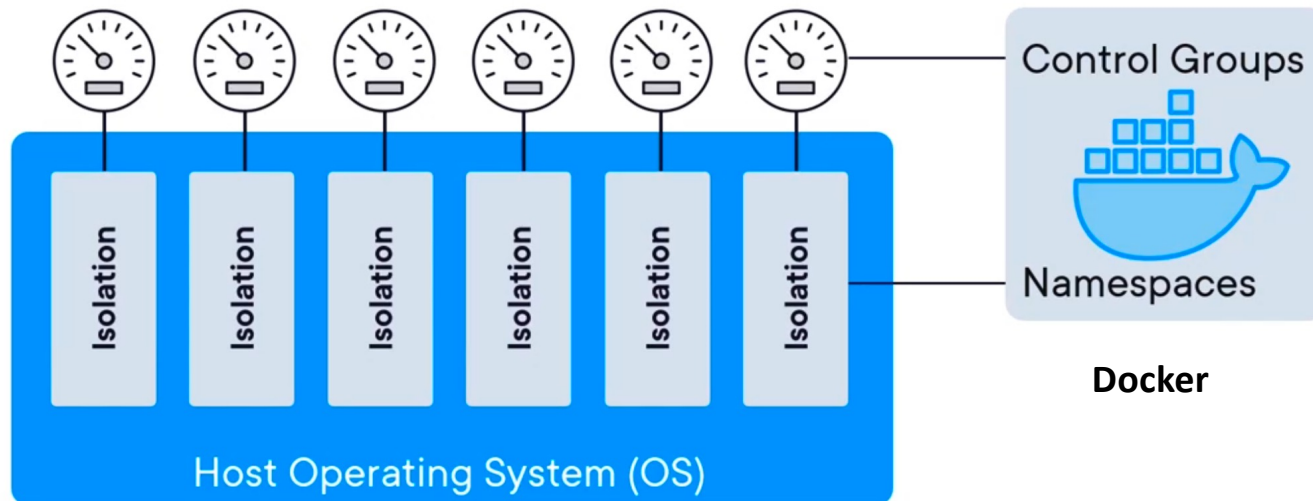
Isolated area of OS with resource limits applied.



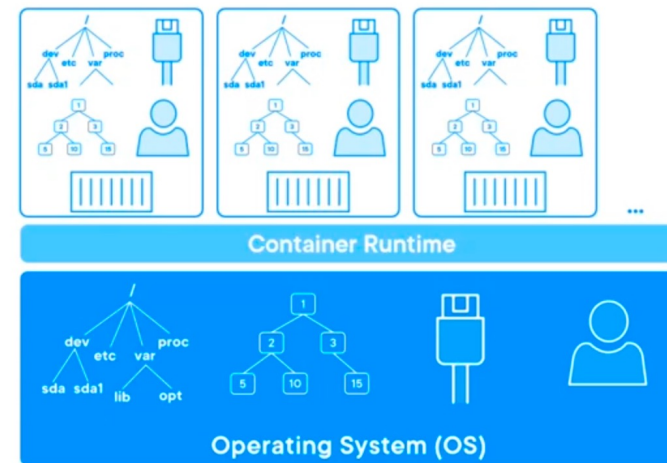
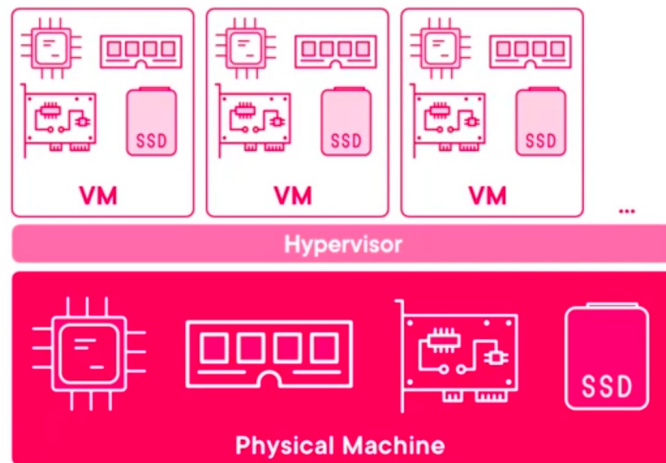
Docker - Principe

Container

Isolated area of OS with resource limits applied.



Docker - Principe



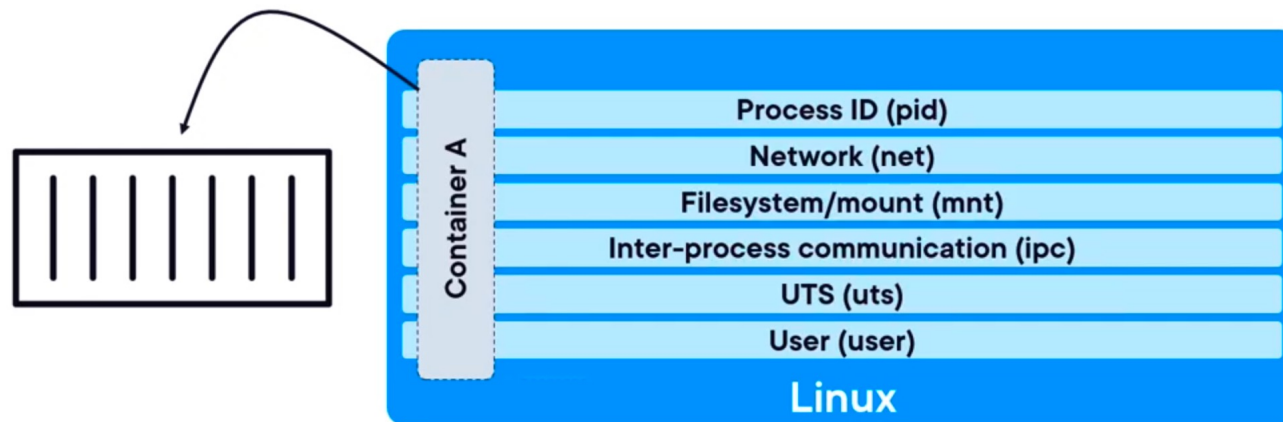
Namespaces

All about isolation

Control Groups

All about limits

Docker - Principe



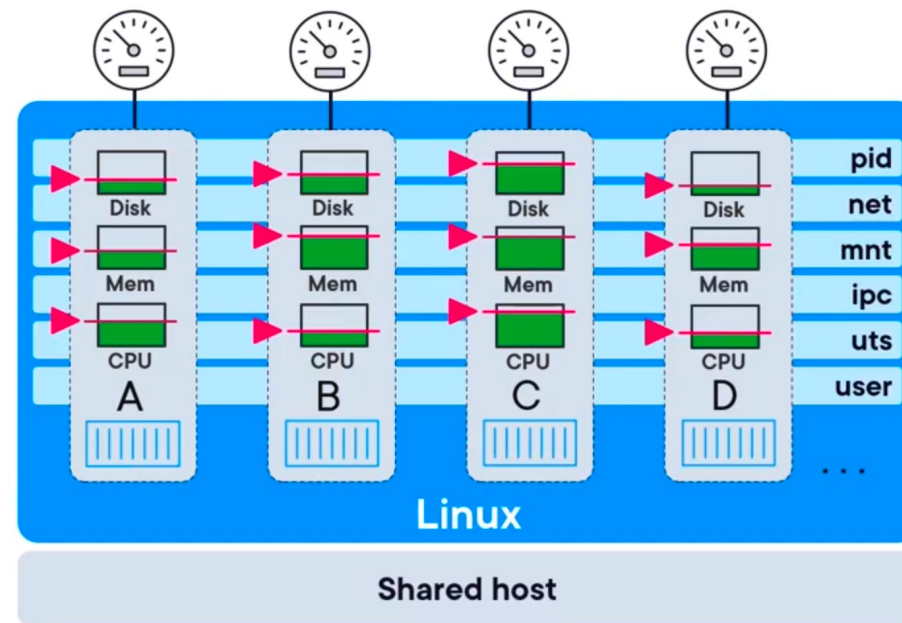
Namespaces

All about isolation

Control Groups

All about limits

Docker - Principe



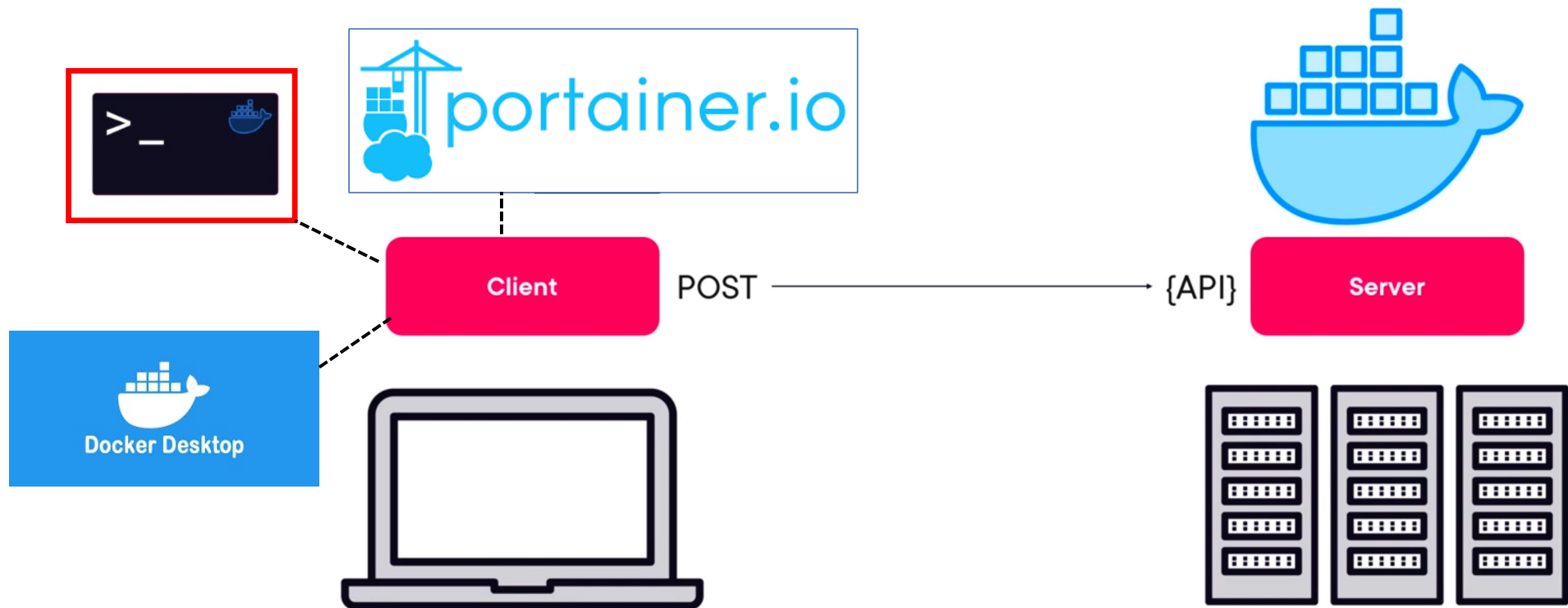
Namespaces

All about isolation

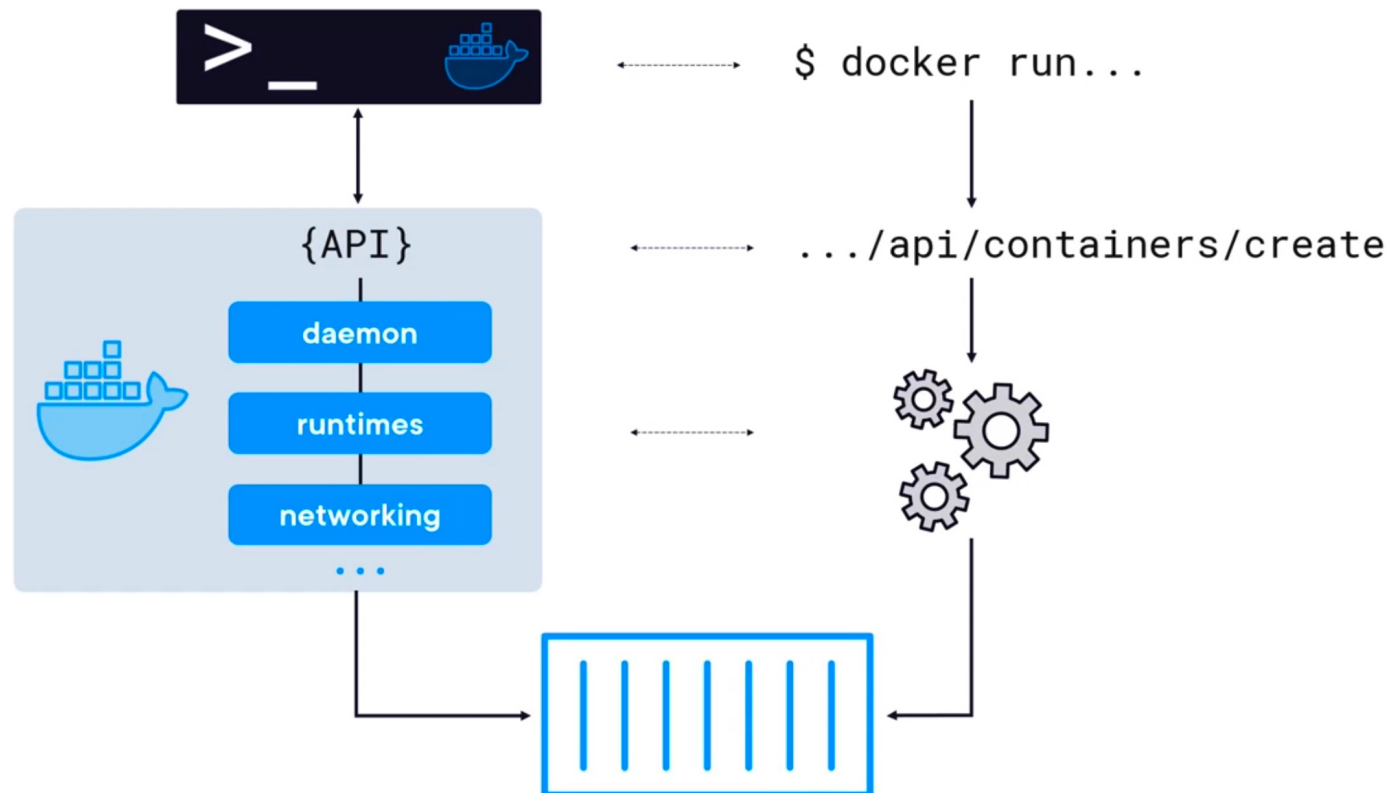
Control Groups

All about limits

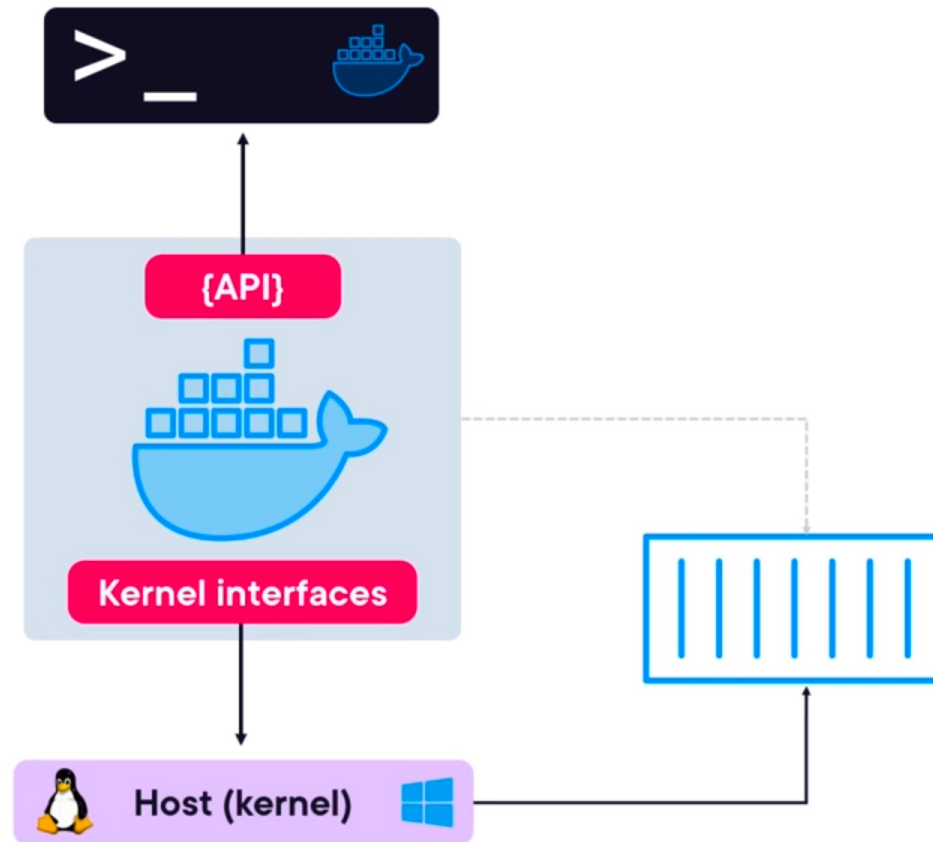
Docker - Principe



Docker - Principe



Docker - Principe



Installer Docker

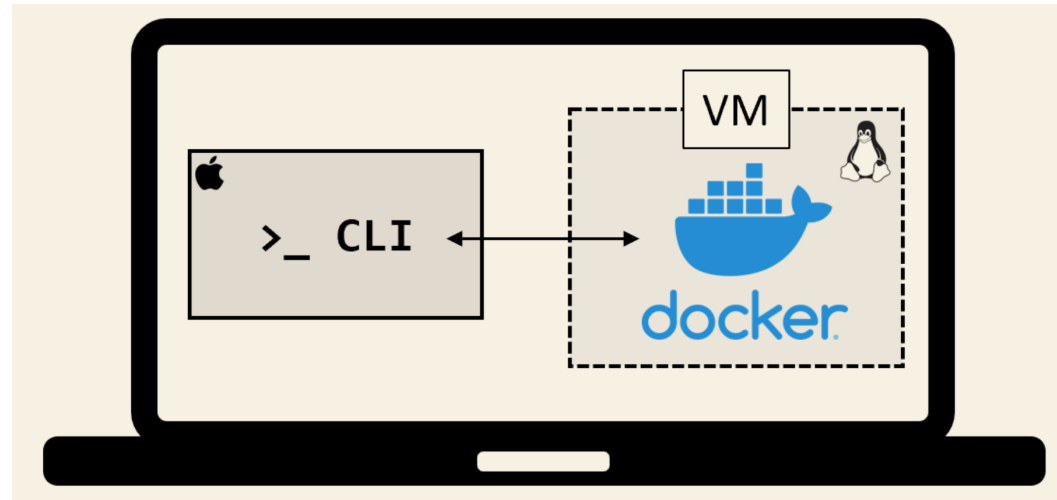
↓ Install Docker Desktop

Install Docker Desktop on [Mac](#),
[Windows](#), or [Linux](#).

```
> docker --version
Docker version 27.2.0, build 3ab4256

~
> docker -v
Docker version 27.2.0, build 3ab4256
```

<https://docs.docker.com/desktop/>



Ecosystème

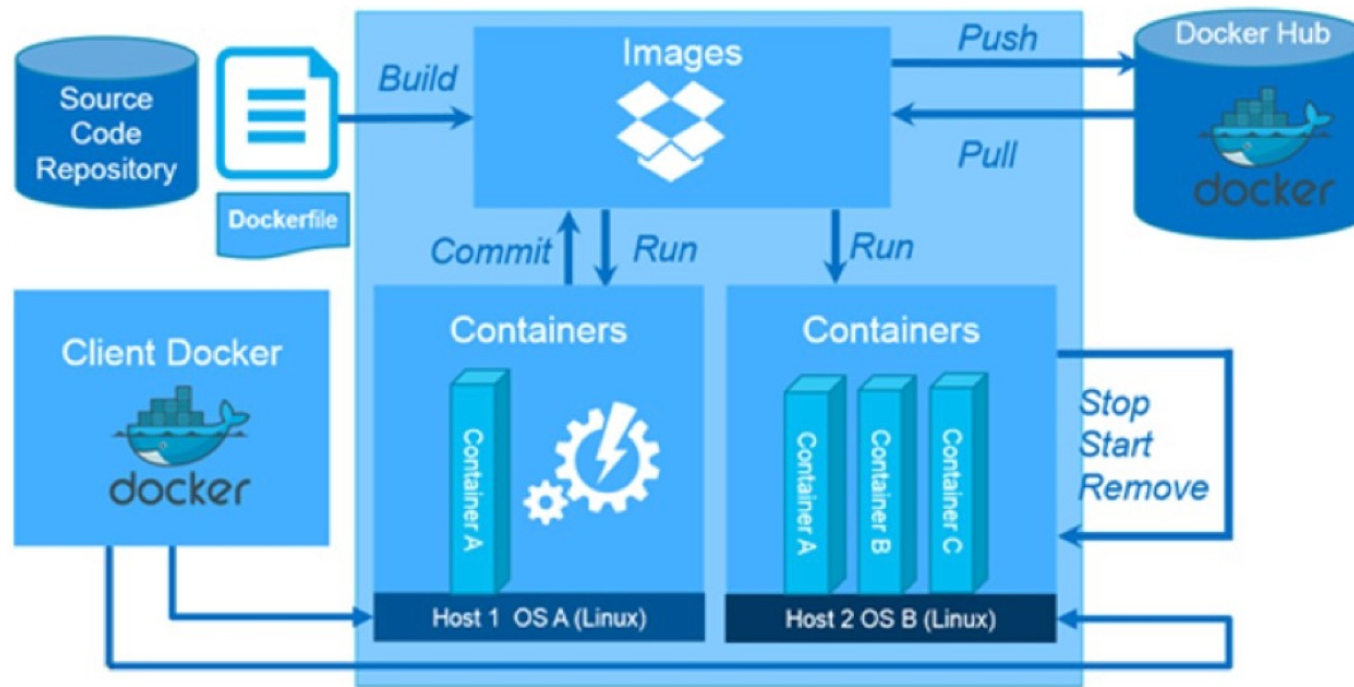


Image vs conteneur

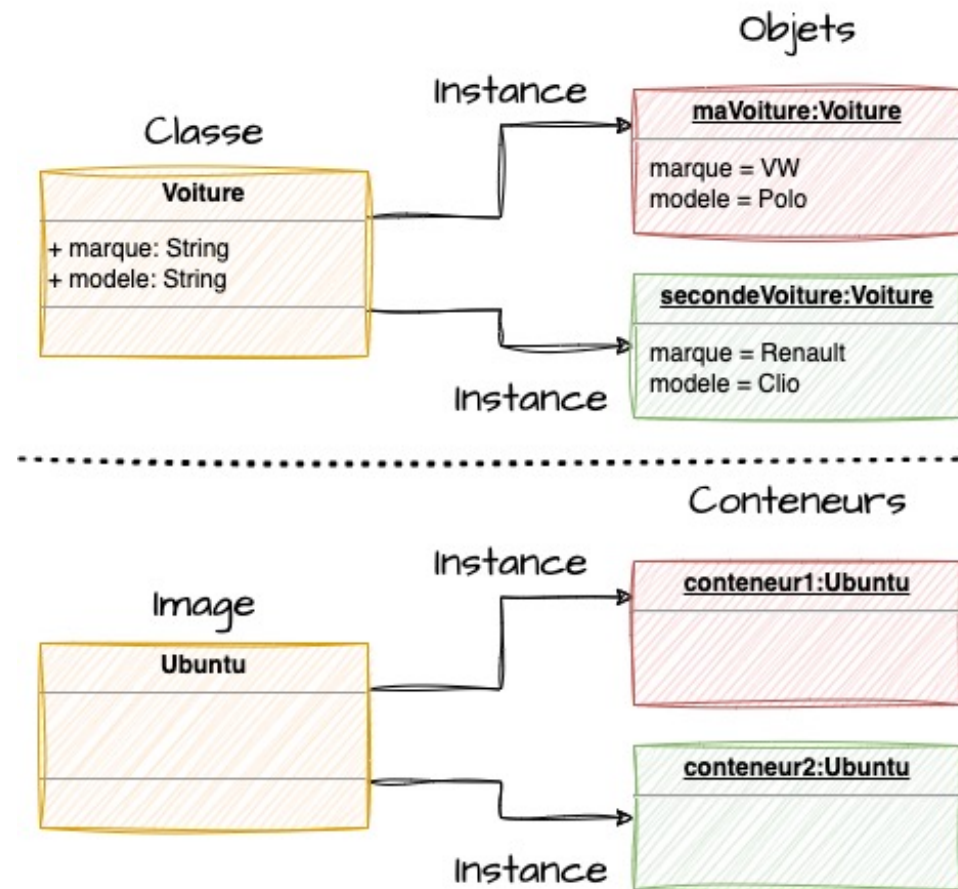


Image vs conteneur

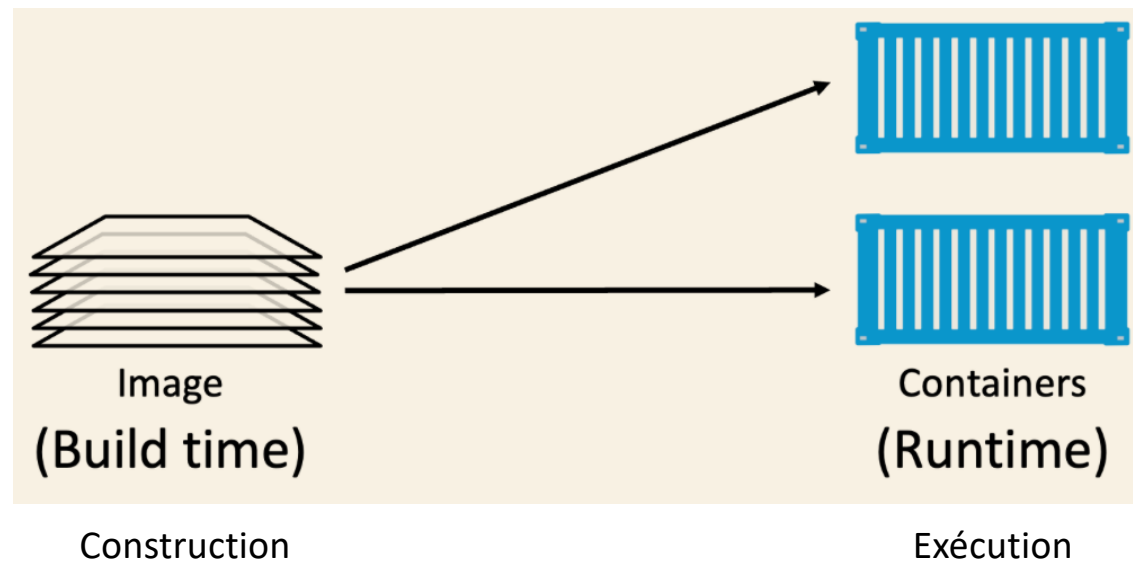


Image vs conteneur



All mean the same

Image

Container image

Docker Image

OCI image



All mean the same

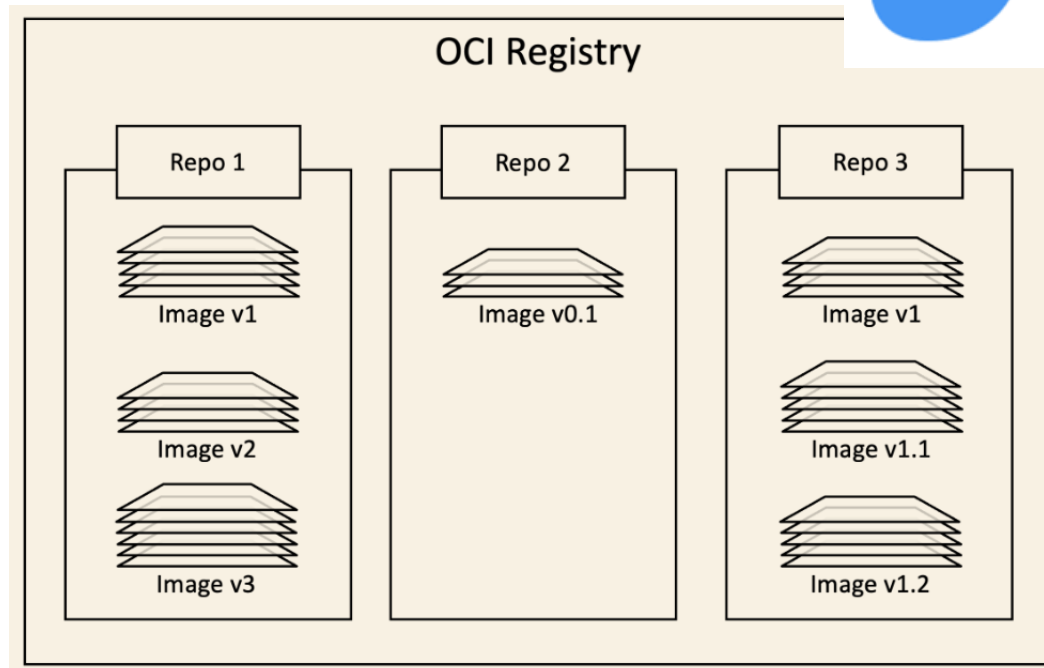
Container

Docker container

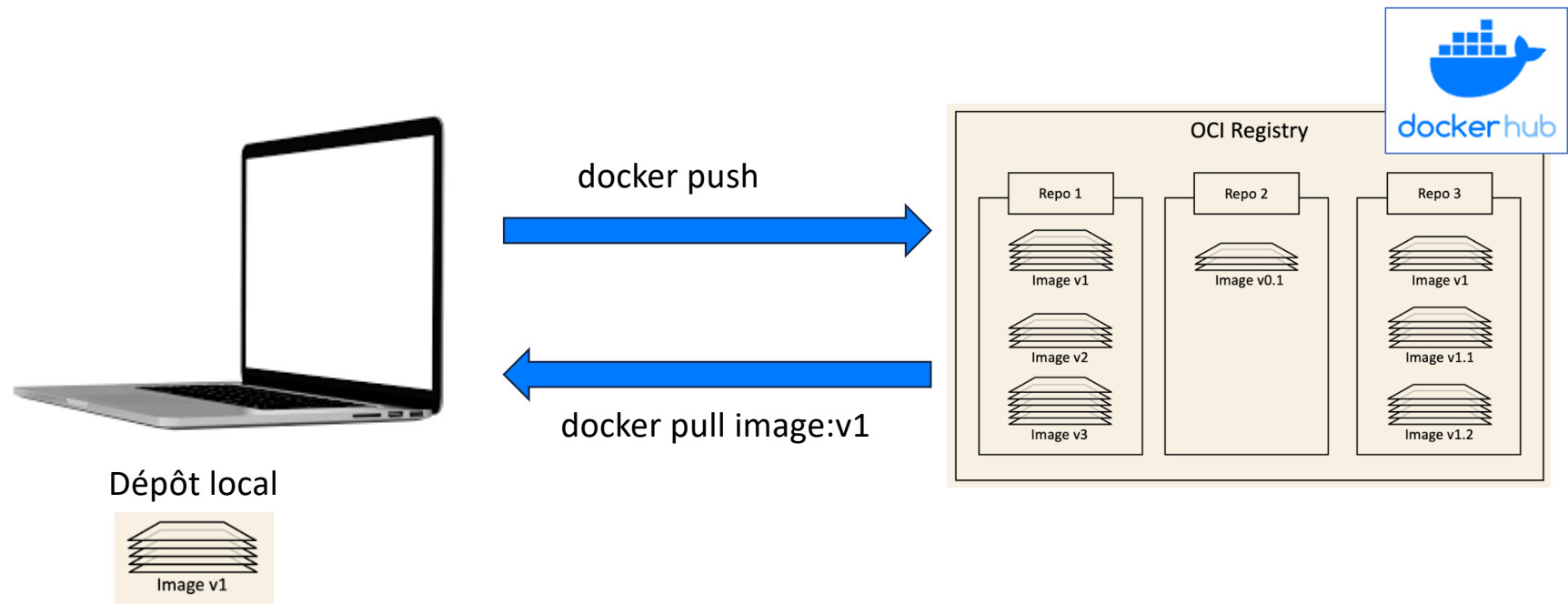
OCI container

Où trouve-t-on des images ?

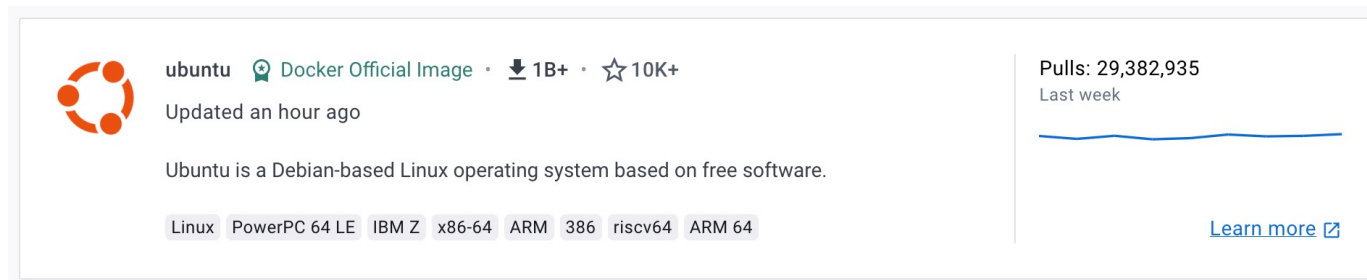
Un registre Docker contient un ou plusieurs dépôts (repository) d'images. Un dépôt renferme une ou plusieurs images.



Comment récupérer une image en local?



Où trouve-t-on des images ?



The screenshot shows the Docker Hub interface for the 'ubuntu' image, which is the 'Docker Official Image'. It features the Ubuntu logo, a description stating it's a Debian-based Linux operating system, and a list of supported architectures: Linux, PowerPC 64 LE, IBM Z, x86-64, ARM, 386, riscv64, and ARM 64. On the right, it displays 'Pulls: 29,382,935' and a line graph for 'Last week'. A 'Learn more' link is also present.

ubuntu  Docker Official Image ·  1B+ ·  10K+

Updated an hour ago

Ubuntu is a Debian-based Linux operating system based on free software.

Linux PowerPC 64 LE IBM Z x86-64 ARM 386 riscv64 ARM 64

Pulls: 29,382,935
Last week

[Learn more](#) 

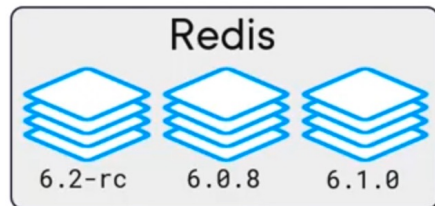
Image Docker Officielle : Elle doit contenir du code :

- à jour
- de haute qualité
- Sécurisé
- bien documenté
- conforme aux meilleures pratiques.

Registre

Official Repos

OCI Registry



E.g. `docker.io/redis:6.0.8`

```
$ Docker pull redis:6.0.8
```

Registre

Unofficial Repos

User/org

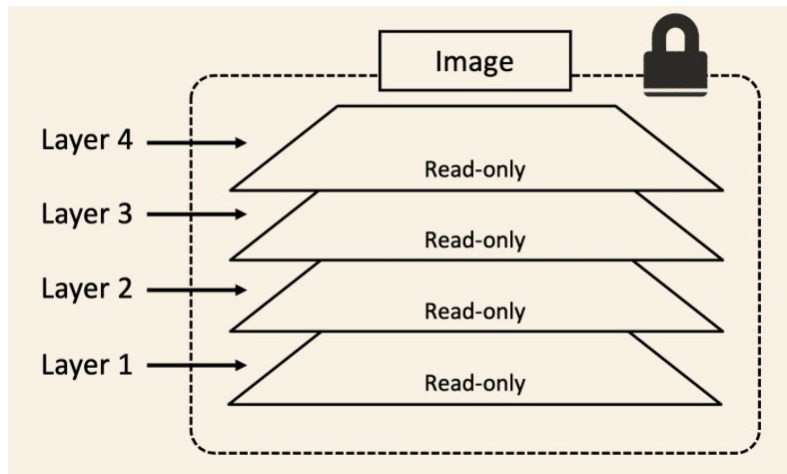
```
$ docker pull bitnami/redis:6.0.8
```

Afficher le contenu du dépôt d'images en local?

```
(base) giannitricarico@MacBook-Pro-de-Gianni ~ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ecommerceapi	latest	4594e6458c5a	6 months ago	621MB
nginx	latest	b21c88e97517	9 months ago	142MB
backup	latest	8c3550ee80a8	9 months ago	4.03MB
mongo	latest	0850fead9327	9 months ago	700MB

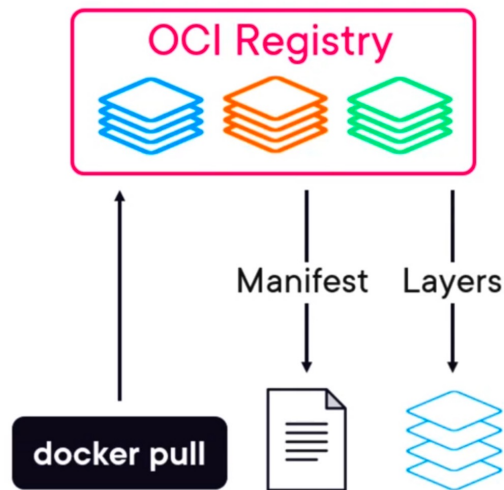
Images



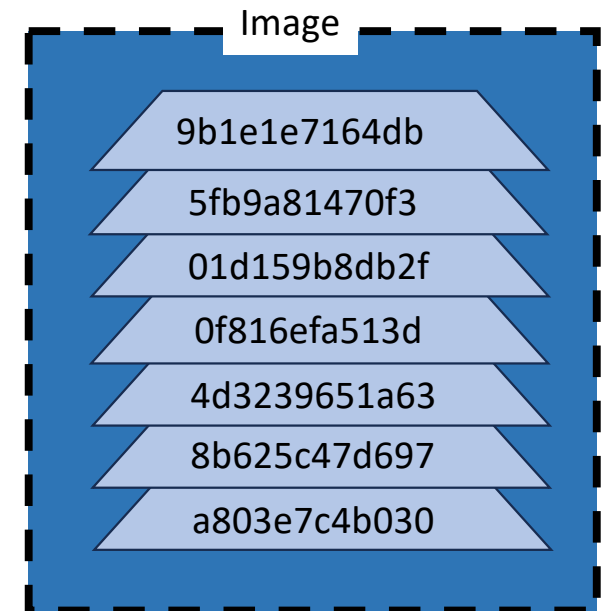
Docker se charge d'empiler les couches et de les représenter comme un seul objet unifié.

Images

```
(base) giannitricarico@MacBook-Pro-de-Gianni ~ docker pull nginx:latest
latest: Pulling from library/nginx
a803e7c4b030: Pull complete
8b625c47d697: Pull complete
4d3239651a63: Pull complete
0f816efa513d: Pull complete
01d159b8db2f: Pull complete
5fb9a81470f3: Pull complete
9b1e1e7164db: Pull complete
Digest: sha256:32da30332506740a2f7c34d5dc70467b7f14ec67d912703568daff790ab3f755
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```



Layers (couches)



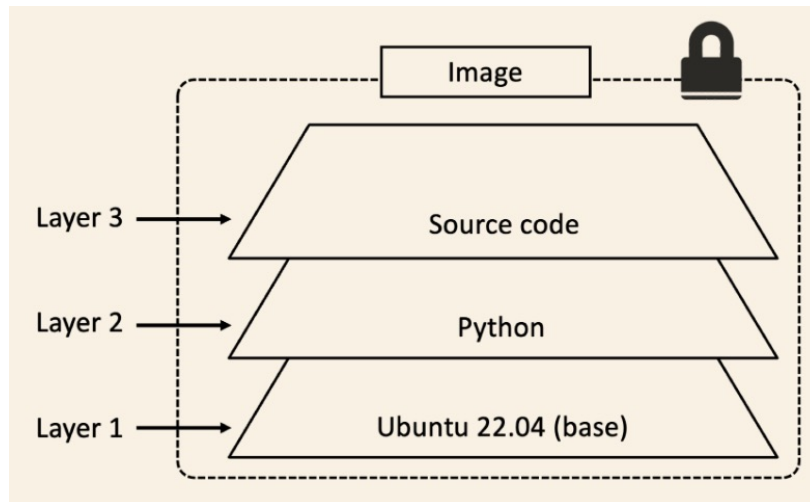
Images

`docker inspect nginx:latest`

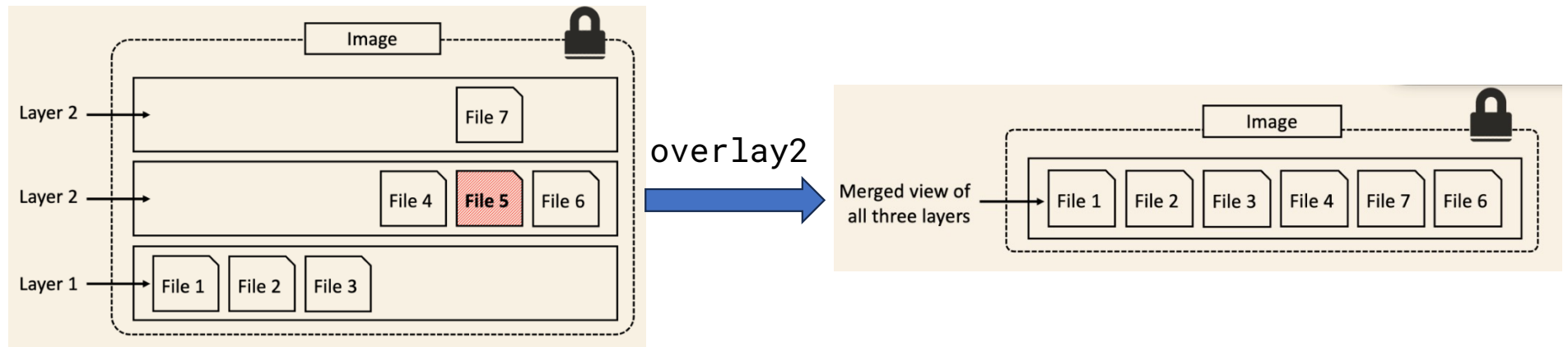
```
    "RootFS": {  
      "Type": "layers",  
      "Layers": [  
        "sha256:d310e774110ab038b30c6a5f7b7f7dd527dbe527854496bd30194b9ee6ea496e",  
        "sha256:eb7e3384f0ab3a08dd0163ab66625d954e511e8fa112c09923e261418c8524cb",  
        "sha256:1dc45c680d0f1615d45021e85028c5d2089468518ec44b7aeeb947cdf0b22364",  
        "sha256:ea43d4f82a03877421d6bed1db76fd58f87555193a7921cefb22d6067d8963a7",  
        "sha256:9c6261b5d19808f0feab896f5f65c0f6287aa8589f880d4af64414608e54c023",  
        "sha256:a7e2a768c198ee94c3a3e8a1cc199a69510da1e39fd85beddc2d4f3200a21d1e",  
        "sha256:d26d4f0eb474290793ba42d20b4880daa7c6ddf0379adeb1c121e04f2840712e"  
      ]  
    },  
    ...  
  },  
  ...  
}
```

Couches + manifest = image

Images - Example



Images – Fusion des couches

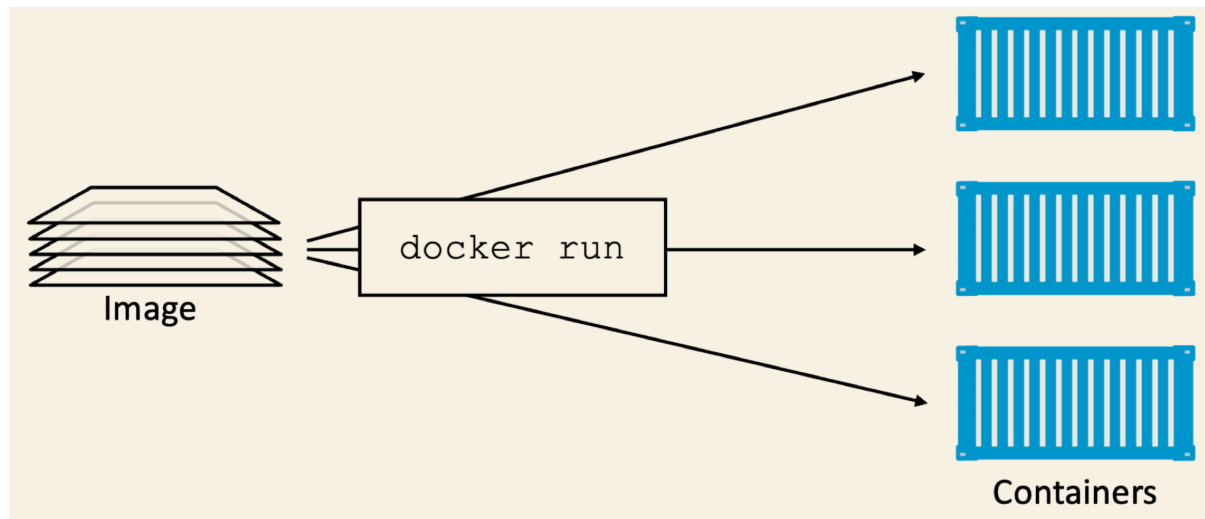


Images – Commande

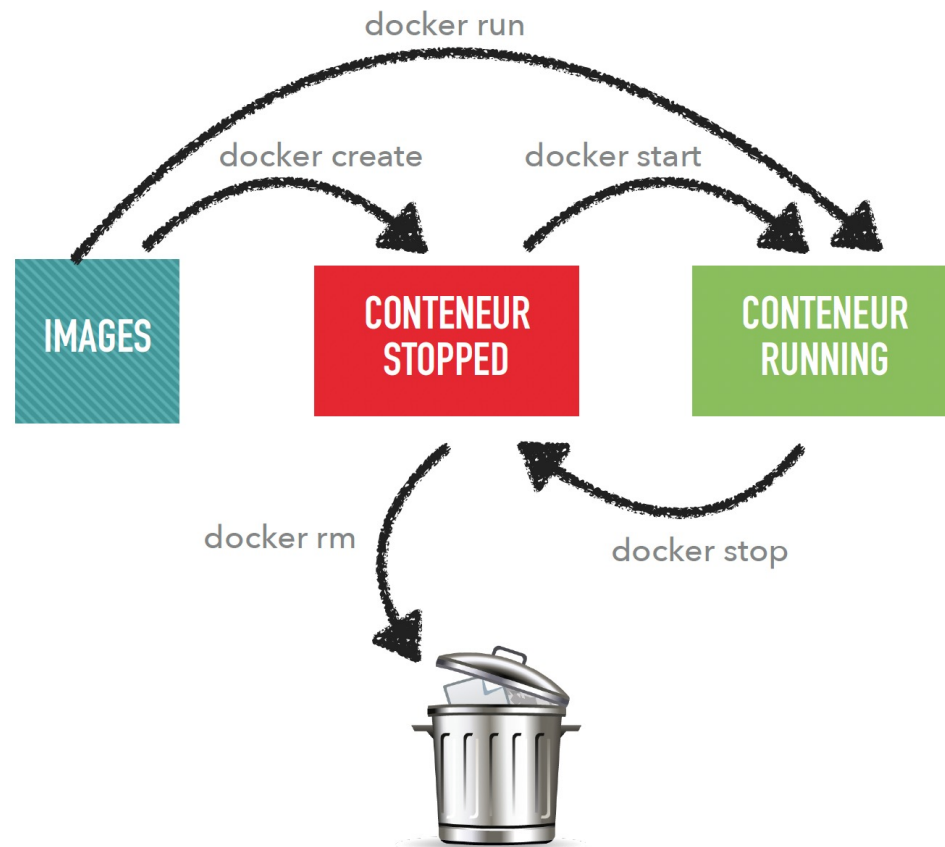
Suppression d'une image

```
$ docker rmi ID
```

Conteneur – Création et exécution



Conteneur – Cycle de vie d'un conteneur



Conteneur - Options

```
$ docker run [OPTIONS] IMAGE:TAG [COMMAND] [ARG...]
```

=> **crée** un conteneur et le **démarre** en une seule opération

Options

-it : permet d'ouvrir un pseudo terminal sur le conteneur -> mode interactif.

--name : permet de nommer le conteneur.

--rm : supprime automatiquement un conteneur qui est l'arrêt.

Conteneur – Commandes systèmes

- `$ docker ps`
 - affiche les conteneurs en cours d'exécution
- `$ docker ps -a`
 - affiche tous les conteneurs (à l'arrêt et à l'exécution)
- `$ docker inspect`
 - affiche toutes les informations liées au conteneur

Conteneur – OS Linux

```
$ docker run [OPTIONS] IMAGE:TAG [COMMAND] [ARG...]
```

Exemples :

```
$ docker run alpine:latest sleep 5
```

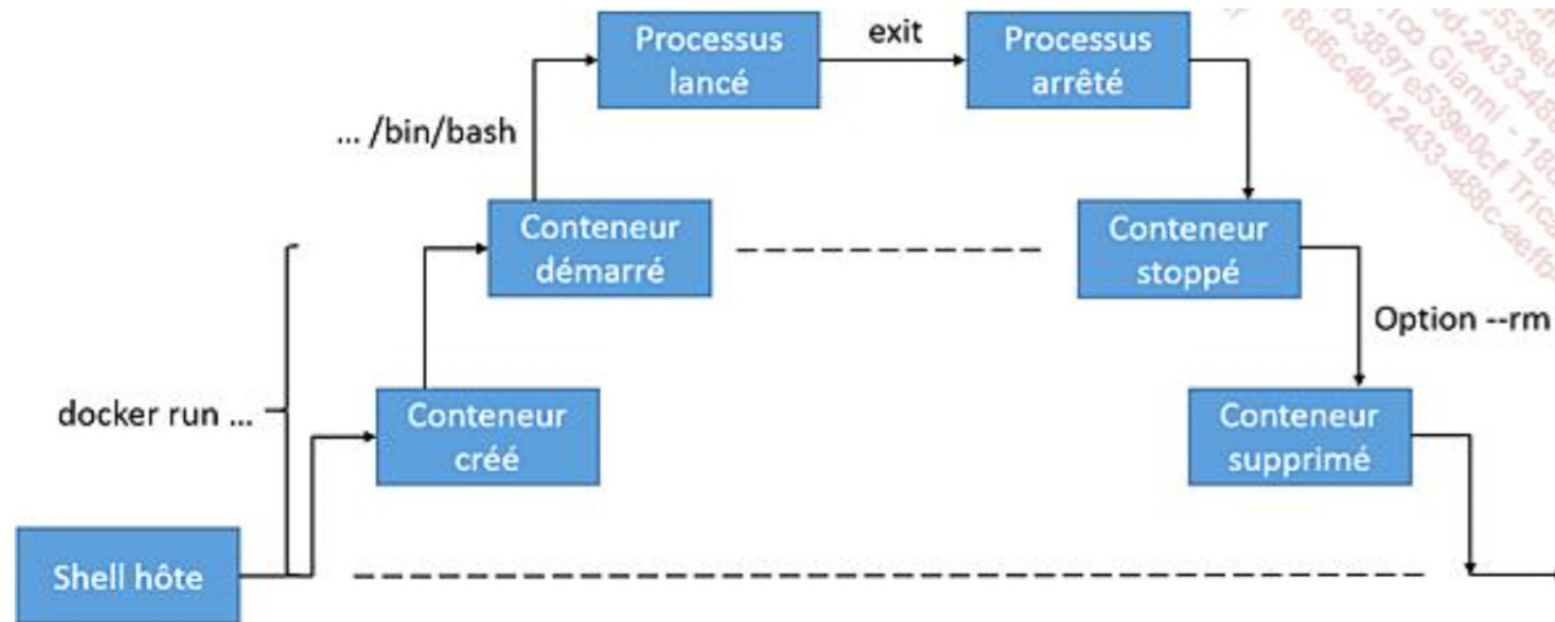
```
$ docker run --rm -it alpine sh
```

```
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # exit
```

```
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # exit
~ took 4m35s
> █
```


Conteneur – Serveur web

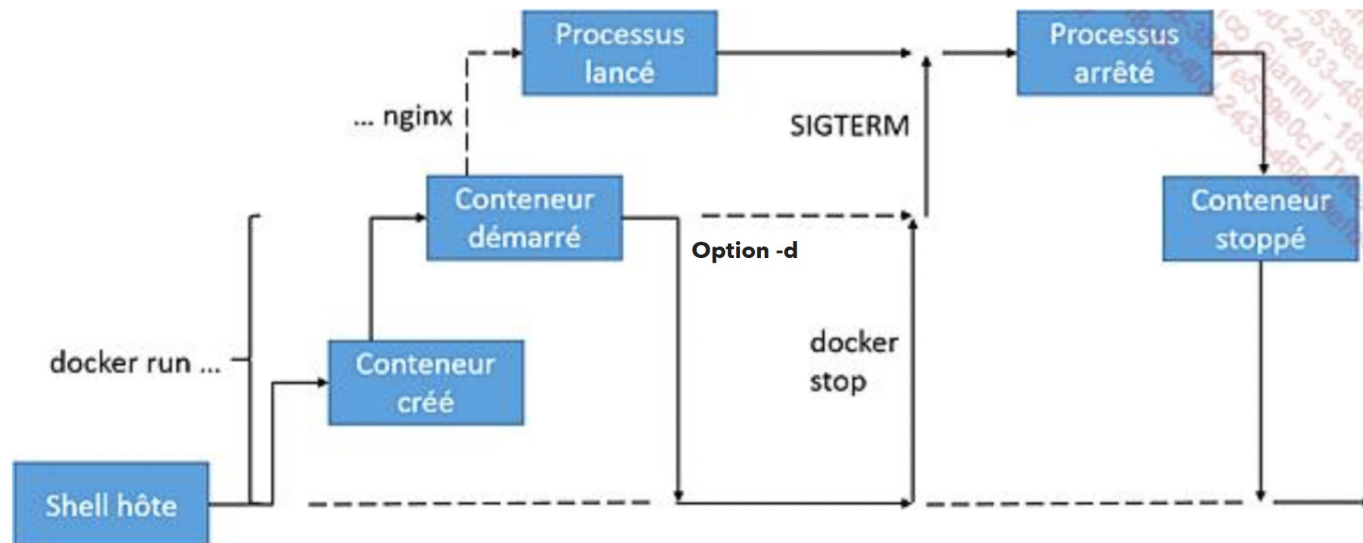
Mode bloquant (hôte)



```
$ docker run nginx
```

Conteneur – Serveur web

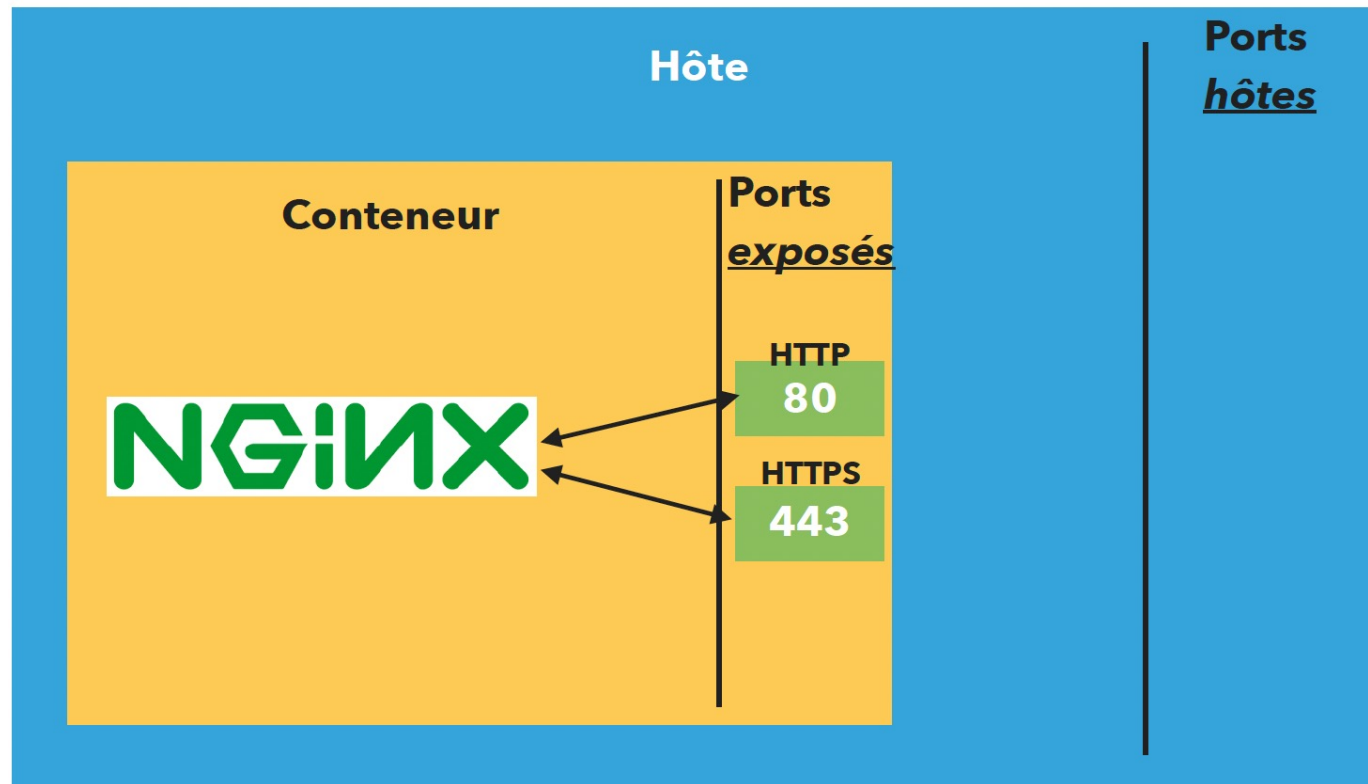
Mode non bloquant (hôte)



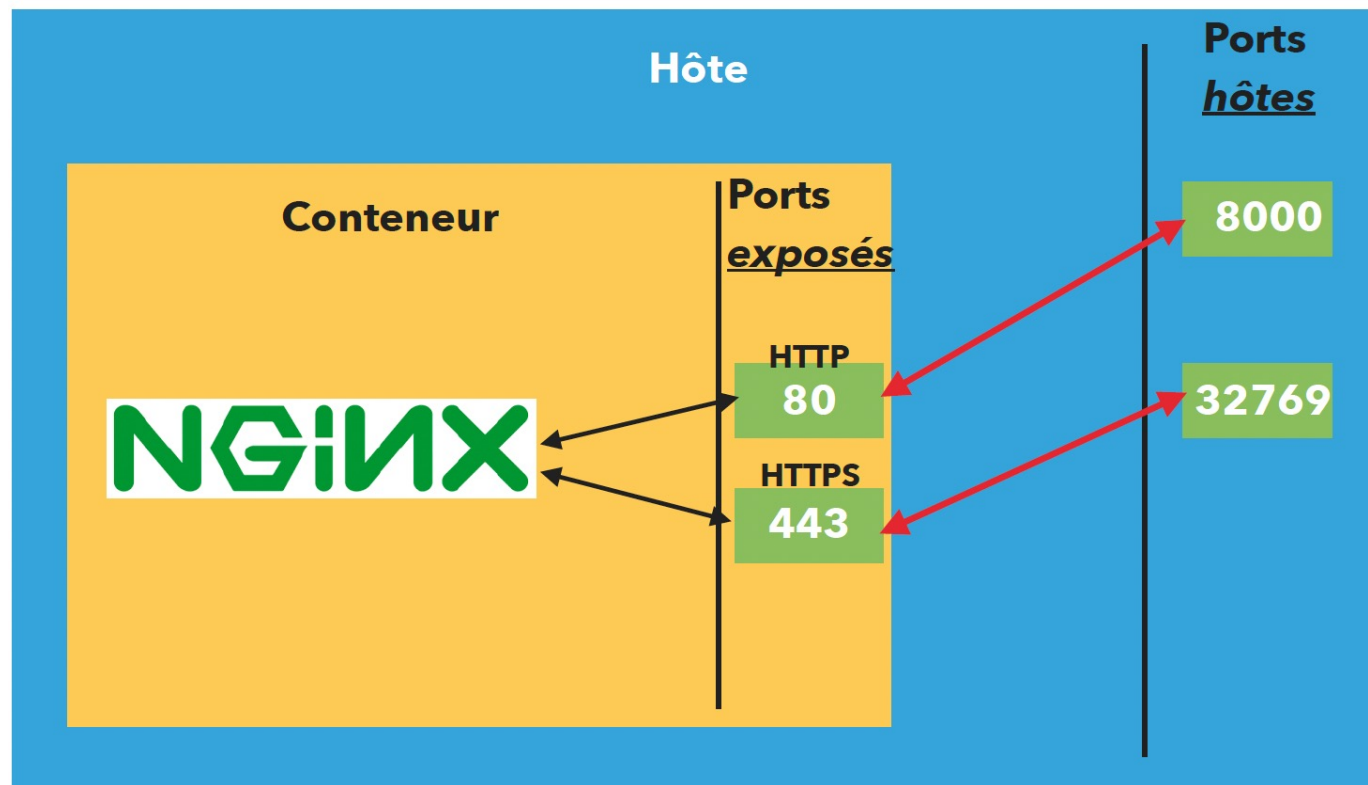
```
$ docker run -d nginx
```

- d : processus daemon (arrière-plan) et détaché du terminal local

Conteneur – Serveur web – ports exposés



Conteneur – Serveur web – Mappage



Conteneur – Serveur web – Mappage

Mappage automatique

- Mapper automatiquement tous les ports exposés dans le conteneur.
docker run -P -d nginx

Mappage manuel

- spécifier manuellement quels ports doivent être mappés parmi ceux exposés
- spécifier manuellement les ports hôtes à utiliser

Option -p de la commande docker run

- Sans le port hôte : -p 80 (mappage automatique)
- Avec le port hôte : -p 8000:80 (mappage manuel)

Conteneur – Serveur web

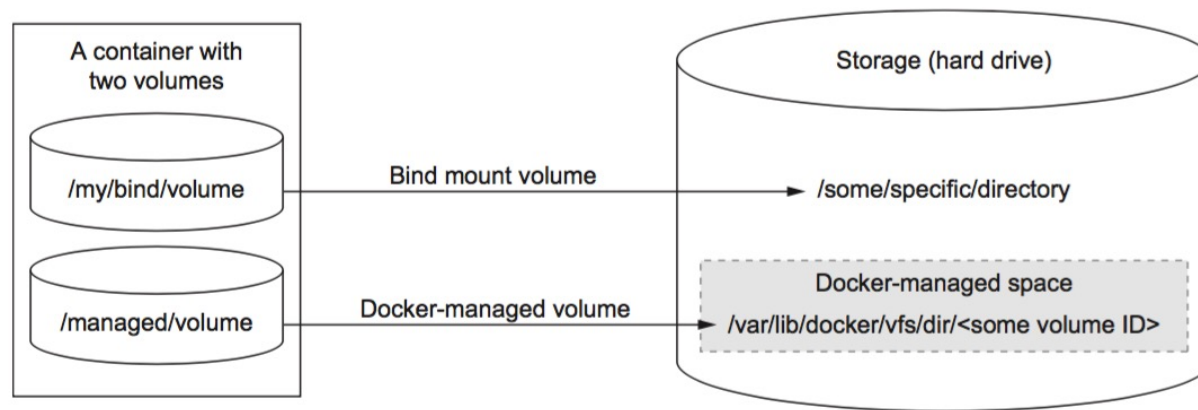
```
docker exec -it nomConteneur /bin/bash
```

=> Permet de lancer un terminal bash dans un conteneur en cours d'exécution.

Conteneur – Volumes

- Type de **dossier spécial** dans un conteneur
- Dossier qui peut être **partagé**
- Volumes permettent de gérer la **persistance** des données même après la suppression du conteneur.

Conteneur – Volumes



Conteneur – Volumes

-v <dossier hôte> : <dossier conteneur>

```
Docker run -d -P -v /Users/gianni/Documents/Labs/WebDocker  
:/usr/share/nginx/html nginx
```

Conteneur – Variables d'environnement

Passer des variables d'environnement à un conteneur.

```
docker run --name some-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -  
d mysql:tag
```

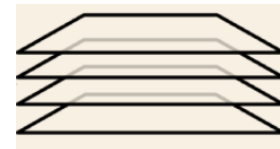
Construction d'images – Conteneur

1) Création d'une image à partir d'un conteneur



`docker commit ID_CONTAINER heh8mons7000/ecommerceapi:1.0`

Nouvelle image



heh8mons7000/ecommerceapi:1.0

Construction d'images – Conteneur

2) Logger sur Docker Hub

```
> docker login
```

USING WEB BASED LOGIN

To sign in with credentials on the command line

Your one-time device confirmation code is: **HDNL**

Press ENTER to open your browser or submit your

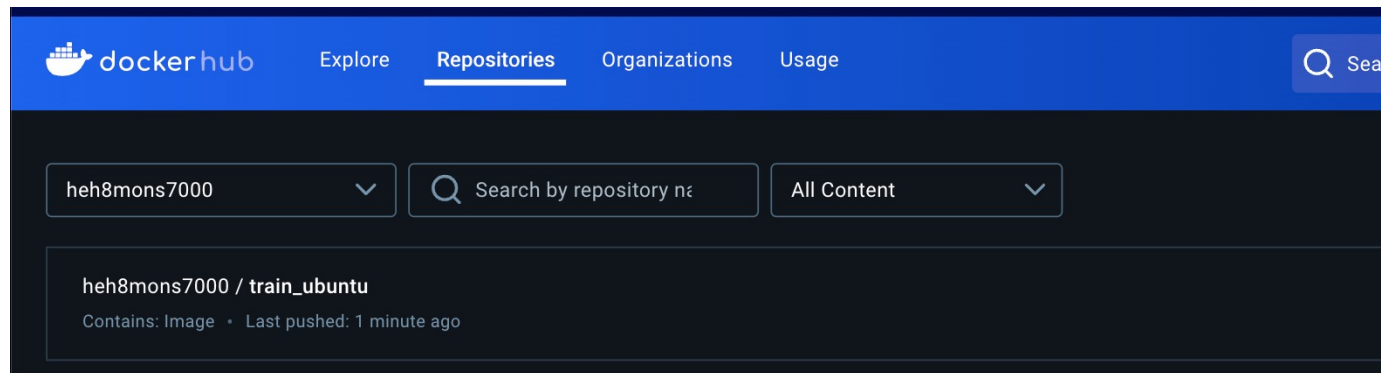
Waiting for authentication in the browser...

Login Succeeded

Construction d'images – Conteneur

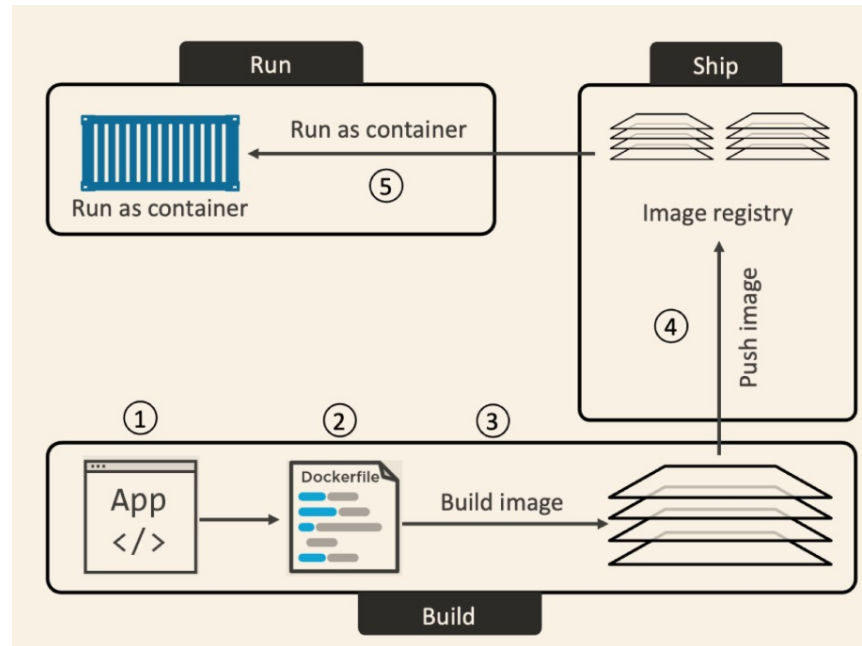
3) Publier son image sur Docker Hub

```
> docker push heh8mons7000/train_ubuntu:1.0
The push refers to repository [docker.io/heh8mons7000/train_ubuntu]
f73d1a40c796: Pushed
a4feb6502043: Mounted from library/ubuntu
1.0: digest: sha256:0eeca2662399ebaa43fa74470c37f80aaf52b24749d1a05791cf5808aed07c4a size: 741
```

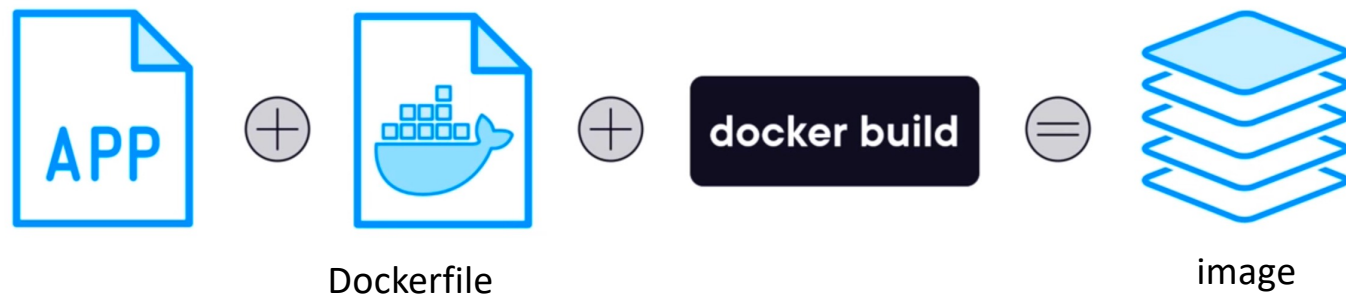


Construction d'images – Dockerfile

Création d'une image à partir d'un fichier Dockerfile



Construction d'images – Dockerfile



Construction d'images – Dockerfile

```
=> [1/3] FROM docker.io/library/ubuntu:24.10  
=> [2/3] RUN apt-get update  
=> [3/3] RUN apt-get install sl -y  
=> exporting to image
```

```
=> [1/3] FROM docker.io/library/ubuntu:24.10  
=> CACHED [2/3] RUN apt-get update  
=> [3/3] RUN apt-get install sl cowsay -y  
=> exporting to image
```

Ajout d'un paquet => pas de mise à jour des dépendances !

Construction d'images – Dockerfile

```
=> => transferring context: 2B  
=> CACHED [1/2] FROM docker.io/library/ubuntu:24.10  
=> [2/2] RUN apt-get update && apt-get install sl cowsay -y  
=> exporting to image  
=> => exporting layers
```

```
=> => transferring context: 2B  
=> CACHED [1/2] FROM docker.io/library/ubuntu:24.10  
=> [2/2] RUN apt-get update && apt-get install sl cowsay nano -y  
=> exporting to image
```

Construction d'images – Exercice



Construction d'images – Déploiement

<https://render.com/>

Create a new Web Service

Connect a Git repository, or use an existing image.

How would you like to deploy your web service?

☒ Build and deploy from a Git repository

Connect a GitHub or GitLab repository.

☐ Deploy an existing image from a registry **ADVANCED**

Pull a public image from any registry or a private image from Docker Hub, GitHub, or GitLab.

Next

Construction d'images – Déploiement

Create a new **Web Service**

Deploy your web service from an image hosted on a public or private registry.

Deploy an image

Image URL

The image URL for your external image.



docker.io/heh8mons7000/app1:12



Credential (Optional)

Use a credential to access private images.
Manage credentials in [Settings](#).

No credential



Next

Registry Credentials

We currently support [private images](#) from
GitHub, GitLab, and Docker Hub registries.
We don't restrict public image registry types.

This is a public image that can be accessed
without credentials.

Construction d'images – Déploiement

Name
A unique name for your web service.

api_rest_test

Region
The [region](#) where your web service runs.

Frankfurt (EU Central) ▾

Please [enter your payment information](#) to select an instance type with higher limits.

Instance Type	RAM	CPU	Price
☒ Free	512 MB	0.1 CPU	\$0 / month

Construction d'images – Déploiement

WEB SERVICE

api_rest_test

Image

Free

Connect

Manual Deploy

heh8mons7000 / app1

12

https://api-rest-test-m6t3.onrender.com

Events

Logs

Disks

Environment

Shell

Previews

Jobs

Metrics

Scaling

Settings

Free instance types will spin down with inactivity. Upgrade to a paid instance type to prevent this behavior. Learn more.

November 16, 2023 at 11:00 AM

Live

Deploy for 586b27d

12

Search logs

Search

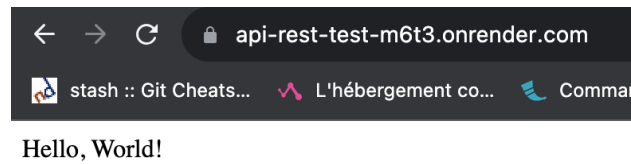
Maximize

Scroll to top

```
Nov 16 11:00:03 AM ==> Starting service...
Nov 16 11:00:35 AM * Debug mode: off
Nov 16 11:00:35 AM WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
Nov 16 11:00:35 AM * Running on all addresses (0.0.0.0)
Nov 16 11:00:35 AM * Running on http://127.0.0.1:5001
Nov 16 11:00:35 AM * Running on http://10.217.211.93:5001
Nov 16 11:00:35 AM Press CTRL+C to quit
Nov 16 11:00:39 AM Your service is live 🎉
Nov 16 11:00:40 AM 127.0.0.1 - - [16/Nov/2023 10:00:40] "GET / HTTP/1.1" 200 -

```

Construction d'images – Déploiement



Acronyme YAML



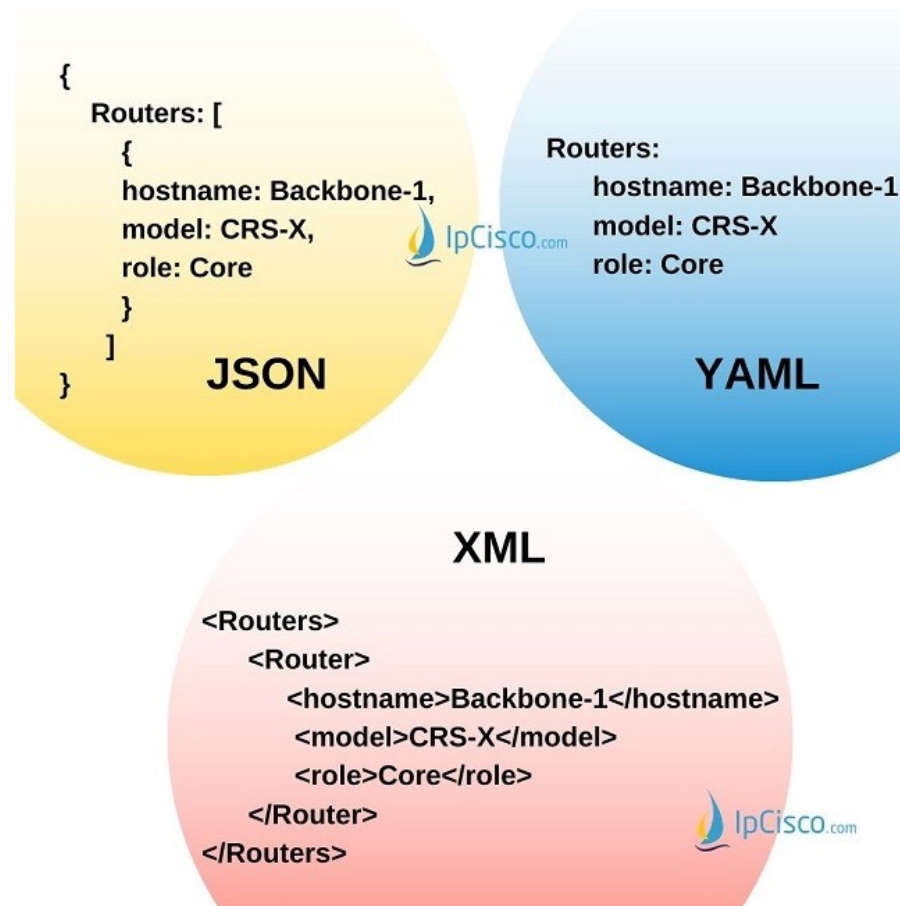
YAML Ain't Markup Language (YAML n'est pas un langage de balisage), ce qui souligne que le langage YAML s'utilise pour représenter des données plutôt que des documents.

YAML

Attribut	YAML	JSON
Verbosité	Moins verbeux	Plus verbeux
Types de données	Prend en charge les types de données complexes.	Ne prend pas en charge les types de données complexes.
commentaires	Prend en charge l'écriture de commentaires en utilisant '#'. 	Ne prend pas en charge la rédaction de commentaires.
Lisibilité	Plus lisible par l'homme.	Moins lisible par l'homme.
Auto-références	Prend en charge le référencement des éléments dans les mêmes documents en utilisant «&» et *.	Ne prend pas en charge l'auto-référencement.
Documents multiples	Prend en charge plusieurs documents dans un seul fichier.	Prend en charge un seul document dans un seul fichier.

https://fre.myservername.com/yaml-tutorial-comprehensive-guide-yaml-using-python#Benefits_Of_Using_A_YAML_File

YAML



<https://ipcisco.com/lesson/data-serialization-languages-json-yaml-xml/>

YAML

- YAML utilise une indentation de style Python pour structurer les données.
- Les caractères de tabulation n'étant pas autorisés, on utilise des espaces blancs.
- Les fichiers YAML portent l'extension .yaml ou .yml.
- Un fichier YAML est structuré sous la forme d'un mappage(dictionnaire) ou d'une liste(tableau).

Comment les données sont stockées dans YAML

YAML peut contenir différents types de blocs de données :

- **Séquence** : valeurs répertoriées dans un ordre spécifique. Une séquence commence par un tiret et un espace (-). Vous pouvez considérer une séquence comme une **liste Python**.
- **Mapping** : paires de clés et de valeurs. Chaque clé doit être unique et l'ordre n'a pas d'importance. Pensez à un **dictionnaire Python**.

Syntaxe de base – Séquence/ tableau/ list

[item1, item2, item3]

Index	Item
0	item1
1	item2
2	item3

The screenshot displays a sequence editor interface. At the top, a tab labeled 'sequence.yml' is visible. Below the tab, the sequence is defined as follows:

```
! sequence.yml > [abc] 2  
1 - item1  
2 - item2  
3 - item3
```

The items are listed with indices 1, 2, and 3. The item 'item3' is highlighted with a light blue background. Below the sequence editor, a 'STRUCTURE' panel is shown, which lists the indices of the items:

- [abc] 0
- [abc] 1
- [abc] 2

The 'STRUCTURE' panel also includes icons for adding, refreshing, and more options.

Syntaxe de base – Mapping/ dictionnaire/ clé-valeur

Key	Value
version	1.2
format	YAML
list	

→

java
javascript
pyhton

```
! mapping.yml ×
! mapping.yml > [ ] list > abc 2
1  version: 1.2
2  format: YAML
3  list:
4    - java
5    - javascript
6    - pyhton
```

STRUCTURE

- # version 1.2
- format YAML
- [] list
 - 0
 - 1
 - 2

Scalaire

Type de données :

- Chaîne de caractères
- Nombre
- Date
- Booléen

! scalar.yml > ...

```
1 float: 1.5
2 name: toto
3 hexadecimal: 0xABC
4
5 boolean: true
6 vide: null
7 date: 2022-11-12
```

STRUCTURE

float 1.5

abc name toto

hexadecimal 2748

boolean true

vide null

abc date 2022-11-12

Document

Plusieurs documents dans un fichier.

```
! documents.yml > ...  
1  language: yaml  
2  version: 1.2  
3  ---  
4  languages:  
5    - java  
6    - javascript  
7  ---  
8  number: 5
```

Multi-lignes

YAML ✓

```
1 multi-line-feed: |  
2   line1  
3   line2  
4   line3  
5  
6 multi-line: >  
7   line1  
8   line2  
9   line3
```

JSON

```
1 {  
2   "multi-line-feed": "line1\nline2\nline3\n",  
3   "multi-line": "line1 line2 line3"  
4 }
```

Séquences imbriquées

! nestedSequ.yml > [] 1 > abc 2

```
1 -  
2 | - java  
3 | - javascript  
4 | - python  
5 -  
6 | - campus technique  
7 | - isims  
8 | - departement des sciences et technologies
```

STRUCTURE

✓ [] 0

abc 0

abc 1

abc 2

✓ [] 1

abc 0

abc 1

abc 2

Mappages imbriquées

! nestedMap.yml > {} person > {} address >  city

```
1  person:
2    name: toto
3    age: 20
4    address:
5      streetname: Av. V. Maistriau
6      number: 8
7      zipcode: 7000
8    city: Mons
```

✓ STRUCTURE

✓ {} person

 name toto

age 20

✓ {} address

 streetname Av. V. Maistriau

number 8

zipcode 7000

 city Mons

Combinaison mappages - séquence

```
! combMapSeq.yml > [ ] persons > {} 1 > {} address > # number
```

```
1  persons:
2    - name: toto
3      age: 20
4      address:
5        streetname: Av.V. Maistriau
6        number: 8
7    - name: titi
8      age: 30
9      address:
10       streetname: Av.V. Maistriau
11       number: 12
```

▼ STRUCTURE

▼ [] persons

▼ {} 0

abc name toto

age 20

▼ {} address

abc streetname Av.V. Maistriau

number 8

▼ {} 1

abc name titi

age 30

▼ {} address

abc streetname Av.V. Maistriau

number 12

Template

```
! template.yml > {} config_prod
```

```
1  config: &db #template
2    database_name: dbname
3    password: 1234test
4
5  config_prod: *db
```

✓ STRUCTURE

✓ {} config

☐ database_name dbname

☐ password 1234test

✓ {} config_prod

☐ database_name dbname

☐ password 1234test

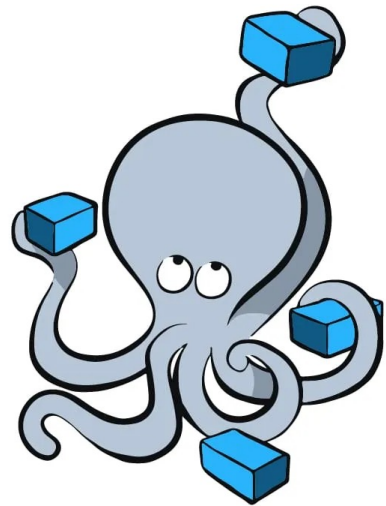
Template

YAML ✓

```
1 config: &db #template
2   database_name: dbname
3   password: 1234test
4
5 config_prod:
6   << : *db
7   password: 1234prod
```

JSON

```
1 {
2   "config": {
3     "database_name": "dbname",
4     "password": "1234test"
5   },
6   "config_prod": {
7     "database_name": "dbname",
8     "password": "1234prod"
9   }
10 }
```



docker
Compose

Docker compose

```
docker run -d -p 8080:8080 -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e  
KC_BOOTSTRAP_ADMIN_PASSWORD=admin --name iam quay.io/keycloak/keycloak:26.0.0 start-dev
```

CLI

docker-compose.yml

```
version: "3"  
services:  
  iam:  
    image: quay.io/keycloak/keycloak:26.0.0  
    container_name: iam  
    environment:  
      - KC_BOOTSTRAP_ADMIN_USERNAME=admin  
      - KC_BOOTSTRAP_ADMIN_PASSWORD=admin  
    command:  
      - start-dev
```

docker compose up